UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

SISTEMA DE GESTIÓN ESCOLAR

MANUAL TÉCNICO

Arquitectura, instalación, persistencia y mantenimiento

Proyecto	Sistema de Gestión Escolar (SGE)
Curso	Programación II
Equipo	Franklin Tilóm Angel Tot Iván Castro
Versión del documento	1.0
Fecha de emisión	5 de noviembre de 2026

Documento elaborado a partir de la versión actualizada de la rama principal del repositorio f-mael/sge-umg.

Control del documento

Elemento	Descripción
Identificador	SGE-MAN-001
Versión	1.0
Estado	Emitido
Audiencia	Desarrolladores, personal de soporte, evaluadores técnicos y administradores del entorno.
Base de elaboración	Rama main del repositorio f-mael/sge-umg
Tecnologías cubiertas	Java 21, JavaFX 21, Maven, JDBC, PostgreSQL 16 y Docker Compose

Contenido

1. Descripción técnica

2. Arquitectura

3. Tecnologías y estructura

4. Ejecución e inicialización

5. Modelo de datos

6. Persistencia JDBC

7. Integración de módulos

8. Construcción, pruebas y despliegue

9. Respaldo y recuperación

10. Mantenimiento, seguridad y Git

1. Descripción técnica

SGE es una aplicación de escritorio JavaFX con interfaz declarada en FXML, lógica de presentación en controladores, modelos de dominio POO y repositorios JDBC conectados a PostgreSQL. Docker Compose provee una instancia reproducible de la base de datos.

Objetivos técnicos

- Separar presentación, control, dominio y persistencia.
- Mantener integridad referencial y reglas de negocio en Java y PostgreSQL.
- Persistir todas las operaciones CRUD.
- Facilitar un entorno local uniforme mediante Docker.

2. Arquitectura

Capa	Responsabilidad	Componentes
Presentación	Controles, formularios y tablas de la interfaz.	FXML y CSS en resources/gt/edu/umg/gestionescolar/view y styles.
Control	Eventos, validación de interacción y enlace con repositorios.	PersonasController, AcademicoController, AsistenciaController, BoletaController.
Dominio	Entidades y relaciones del negocio escolar.	Persona, Estudiante, Docente, Encargado, Curso, CicloEscolar, GradoSeccion, Inscripcion, Asistencia, BoletaCalificaciones, DetalleBoleta.
Persistencia	CRUD, búsqueda, mapeo ResultSet y transacciones.	Repository<T>, repositorios JDBC y DatosCompartidos.
Datos	Esquema relacional y restricciones.	PostgreSQL 16: diez tablas de aplicación.

Flujo principal: FXML → Controller → Repository JDBC → DatabaseManager → PostgreSQL. Los resultados regresan como objetos de dominio para poblar controles y tablas.

3. Tecnologías y estructura

Componente	Versión / uso
Java	21
JavaFX	21.0.2: controls y fxml
Maven	Compilación, dependencias y ejecución
JavaFX Maven Plugin	0.0.8
PostgreSQL JDBC	42.7.2
PostgreSQL	16-alpine en Docker
Docker Compose	Servicio db, volumen persistente y puerto 5432

Estructura relevante

```
sge-umg/
├─ docker-compose.yml
├─ pom.xml
├─ db/schema.sql
└─ src/main/
   ├─ java/gt/edu/umg/gestionescolar/
   │  ├─ App.java
   │  ├─ controller/
   │  ├─ model/
   │  ├─ repository/
   │  └─ util/DatabaseManager.java
   └─ resources/gt/edu/umg/gestionescolar/
      ├─ view/
      └─ styles/
```

4. Ejecución e inicialización

Configuración de conexión

Parámetro	Valor predeterminado
URL	jdbc:postgresql://localhost:5432/gestion_escolar
Usuario	postgres
Contraseña	Variable DB_PASSWORD; si no existe, postgres
Contenedor	sge_postgres
Volumen	sge_pgdata

Secuencia de arranque

1. App.main() inicia JavaFX.
2. App.start() llama DatabaseManager.initializeDatabase().
3. DatabaseManager abre la conexión y ejecuta CREATE TABLE IF NOT EXISTS en orden de dependencias.

4. App verifica la conexión.
5. Se cargan las vistas Personas, Académico, Asistencia y Boletas.
6. Las pestañas académicas y transaccionales se integran en la ventana principal.
7. Al seleccionar pestañas se actualizan listas dependientes.

Comandos

```
git switch main
git pull
docker compose up -d
docker compose ps
mvn clean compile
mvn javafx:run
```

5. Modelo de datos

El esquema puede consultarse en db/schema.sql y también se crea desde DatabaseManager. La siguiente tabla resume claves y restricciones principales.

Tabla	Clave / relaciones	Reglas principales
encargados	PK id	cui único; nombre, apellido, parentesco y dirección obligatorios.
estudiantes	PK id; FK id_encargado	carnet único; fecha de nacimiento obligatoria; encargado puede quedar nulo.
docentes	PK id	codigo_empleado único; especialidad obligatoria.
ciclos_escolares	PK id_ciclo	año y fechas obligatorios; indicador activo.
grados_secciones	PK id_grado_seccion	nombre, nivel y sección obligatorios.
cursos	PK id_curso; FK docente y grado	curso vinculado con docente y grado/sección.
inscripciones	PK id_inscripcion; FK estudiante, grado y ciclo	relaciona estudiante con estructura académica y ciclo.
asistencias	PK id_asistencia; FK estudiante y curso	estado Presente/Ausente; combinación estudiante-curso-fecha única.
boletas_calificaciones	PK id_boleta; FK estudiante y ciclo	promedio entre 0 y 100; estudiante-ciclo-período único.
detalles_boletas	PK id_detalle; FK boleta y curso	nota 0–100; peso > 0; curso no repetido dentro de la boleta.

Relaciones esenciales

- Un encargado puede relacionarse con varios estudiantes.
- Un docente y un grado/sección pueden participar en varios cursos.
- Una inscripción une estudiante, grado/sección y ciclo escolar.
- Una asistencia une estudiante y curso en una fecha.
- Una boleta pertenece a un estudiante y ciclo; contiene múltiples detalles de curso.
- La relación BoletaCalificaciones–DetalleBoleta implementa el patrón maestro–detalle.

Integridad

Las claves foráneas evitan referencias inexistentes; las restricciones UNIQUE impiden duplicados funcionales; CHECK limita estados, notas, promedios y pesos; las acciones ON DELETE aplican CASCADE o SET NULL según el tipo de relación.

6. Persistencia JDBC

Contrato Repository<T>

Operación	Propósito
save(T)	Inserta una entidad.
update(T)	Actualiza por identificador.
delete(int)	Elimina por identificador.
findById(int)	Devuelve Optional<T>.
findAll()	Lista todas las entidades.
search(String)	Consulta por criterio textual o relacionado.

Patrón de implementación

1. Solicitar Connection a DatabaseManager.
2. Preparar SQL parametrizado con PreparedStatement.
3. Asignar valores y ejecutar la operación.
4. Recuperar claves generadas cuando corresponda.
5. Mapear ResultSet a modelos, incluyendo objetos relacionados.
6. Propagar errores como excepciones comprensibles para el controlador.

Transacciones de boletas

BoletaRepository trata la cabecera y los detalles como una unidad. Para guardar o actualizar, desactiva el autocommit, escribe la boleta, procesa detalles y confirma con commit. Ante una falla ejecuta rollback, evitando una cabecera sin detalles o un conjunto parcial de notas.

Cálculo de promedio

BoletaCalificaciones calcula el promedio ponderado a partir de nota y peso. Los repositorios validan notas entre 0 y 100, pesos mayores que cero y ausencia de cursos repetidos antes de persistir.

DatosCompartidos

Centraliza instancias de repositorios para que los controladores consulten la misma fuente PostgreSQL. No representa una caché permanente: la fuente de verdad es la base de datos.

7. Integración de módulos

Módulo consumidor	Datos requeridos
Estudiantes	Encargados
Cursos	Docentes y grados/secciones
Inscripciones	Estudiantes, grados/secciones y ciclo activo
Asistencia	Estudiantes y cursos

Módulo consumidor	Datos requeridos
Boletas	Estudiantes, ciclos y cursos

App registra acciones al cambiar de pestaña para actualizar listas de cursos y ciclos. Los botones “Actualizar listas” permiten una recarga manual cuando los datos base cambian durante la misma sesión.

8. Construcción, pruebas y despliegue local

Preparación

- JDK 21 en JAVA_HOME.
- Maven disponible en PATH.
- Docker Desktop funcionando.
- Puerto 5432 libre.

Compilación y ejecución

```
docker compose up -d
mvn clean compile
mvn javafx:run
```

Pruebas mínimas de aceptación

Caso	Resultado esperado
Arranque con Docker activo	Se abre la ventana y se crean las diez tablas.
CRUD de persona	El registro persiste tras reiniciar.
Curso relacionado	Muestra docente y grado correctos.
Inscripción	Une estudiante, grado y ciclo activo.
Asistencia	Guarda estado y rechaza duplicado diario.
Boleta maestro–detalle	Guarda cabecera, cursos, notas y promedio en una transacción.
Búsqueda	Filtra por los criterios del módulo.
Integridad	Rechaza identificadores duplicados y valores fuera de rango.

Verificación directa en PostgreSQL

```
docker exec sge_postgres psql -U postgres -d gestion_escolar -c "\dt"
```

Para diagnóstico adicional, consulte cada tabla con SELECT y confirme que las claves foráneas correspondan a los objetos visibles en la interfaz.

Despliegue local reproducible

El repositorio, Docker Desktop, JDK y Maven son suficientes. Docker Compose descarga la imagen PostgreSQL, crea la red y el volumen, y expone el puerto. La aplicación ejecuta el esquema idempotente al iniciar.

9. Respaldo y recuperación

Respaldo

```
docker exec sge_postgres pg_dump -U postgres -d gestion_escolar -Fc -f /tmp/sge.dump
docker cp sge_postgres:/tmp/sge.dump ./sge.dump
```

Restauración

```
docker cp ./sge.dump sge_postgres:/tmp/sge.dump
docker exec sge_postgres pg_restore -U postgres -d gestion_escolar --clean --if-exists /tmp/sge.dump
```

PRECAUCIÓN	La restauración con --clean reemplaza objetos existentes. Realice un respaldo reciente y asegúrese de trabajar sobre la base correcta.

Persistencia del volumen

docker compose down detiene y elimina el contenedor, pero conserva sge_pgdata. docker compose down -v también elimina el volumen y, por tanto, los datos locales; úselo únicamente cuando se desee reinicializar por completo.

10. Mantenimiento, seguridad y Git

Diagnóstico frecuente

Síntoma	Causa probable / solución
Docker API no disponible	Docker Desktop cerrado o motor detenido.
Connection refused	Contenedor apagado o puerto incorrecto.
Autenticación fallida	DB_PASSWORD no coincide con PostgreSQL.
Relación no existe	Ejecute la aplicación para inicializar o revise db/schema.sql.
Error FXML	Revise fx:controller, fx:id, onAction y rutas de recursos.
Datos no visibles	Actualice listas y confirme la transacción en PostgreSQL.

Seguridad y evolución

- En producción, no use la contraseña predeterminada; inyecte secretos por variables de entorno.
- No exponga PostgreSQL fuera del equipo sin reglas de red y autenticación adecuadas.
- Mantenga PreparedStatement para evitar inyección SQL.
- Incorpore migraciones versionadas si el esquema continúa evolucionando.
- Agregue pruebas unitarias de modelos y pruebas de integración de repositorios.
- Use logging estructurado y evite imprimir credenciales.

Flujo Git recomendado

1. Actualizar main con git pull.
2. Crear una rama feature/<responsable>-<cambio>.
3. Realizar cambios pequeños y compilar.
4. Ejecutar pruebas funcionales y de persistencia.
5. Crear commits descriptivos y subir la rama.

6. Abrir pull request con cambios y pruebas.
7. Revisar, fusionar y volver a actualizar main.

```
git switch main
git pull
git switch -c feature/nombre-cambio
# editar y probar
git add .
git commit -m "Descripción breve"
git push -u origin feature/nombre-cambio
```

Referencias del proyecto

- Repositorio: https://github.com/f-mael/sge-umg
- Esquema SQL: db/schema.sql
- Configuración: docker-compose.yml y pom.xml
- Punto de entrada: src/main/java/gt/edu/umg/gestionescolar/App.java

Fin del Manual Técnico.